

Data Structures and Algorithms

05.31.2026

Remy Welham

*Comparing and Contrasting
Four Data Structures*

Data Structures and Their Uses

Binary Search Tree

- Binary Search Trees are to organize information as though on branches of a tree. It can have as many branches or nodes as needed.
- Used for hierarchical relations between different nodes
- Efficient and effective at Search and Sorting tasks

Array/Vector

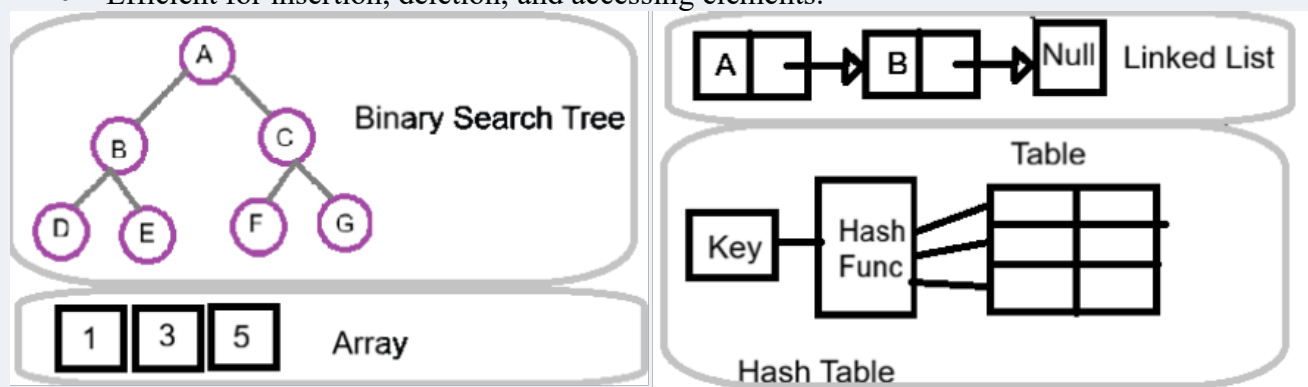
- Arrays and Vectors are lists of data. Arrays typically require a known number of items or a set size, whereas Vectors can modify their length more easily. However, they are very similar for the purposes of this document.
- Used for keeping and making lists
- Efficient storing and retrieving from a list

Linked List

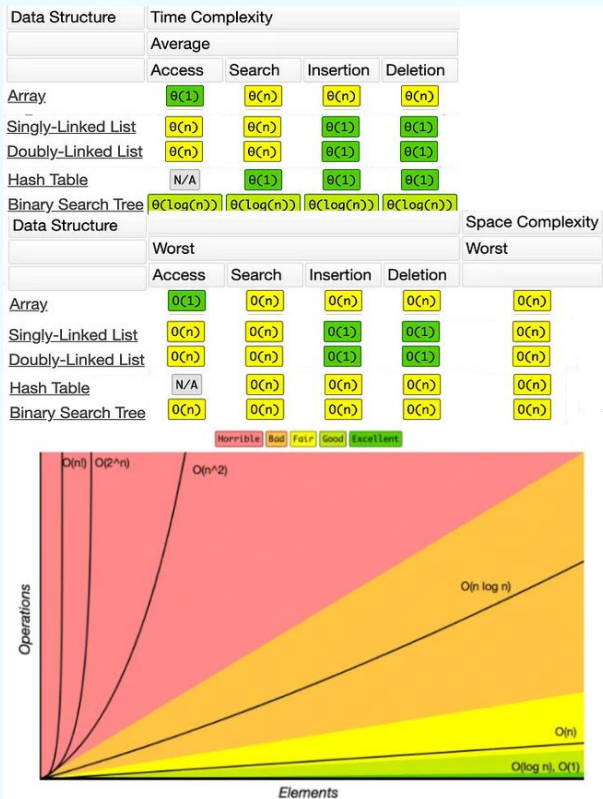
- Linked Lists are lists comprised of an item and a pointer to the next part of the list. They can be modified and inserted as the pointer is adjusted to point to the next item in the list and can be reassigned.
- Used for keeping lists where information will be added or deleted frequently
- Efficient for addition and deletion of data but slower for accessing elements

Hash Table

- Hash Tables are similar to dictionaries in that there is a key (such as a word) that is put through a hash function to store and retrieve elements (such as the definition and origin of the word) from the table.
- Used for secure data handling and large databases
- Efficient for insertion, deletion, and accessing elements.



Timing



Manralai. (2024, September 7). Understanding data structures and time complexity in 5 minutes | by Manralai | Medium. <https://manralai.medium.com/understanding-data-structures-and-time-complexity-in-5-minutes-4e230401a9a0>

So, how does this connect to algorithms and searching? Timing. It all comes down to how many resources and how much time the algorithms take.

All computing takes time and resources from the system. Different data structures use these differently for different tasks.

Big O notation is used for judging time as seen in the graph to the left. The second chart shows the operations to number of elements efficiency with $O(\log n)$ and $O(1)$ being the most efficient and quick.

In best cases, Hash Tables and Binary Search Trees are the most efficient, but why is Access for hash tables N/A? Accessing things from the hash table changes based on more factors than the other data structures due to the nature of using a hash function.

Binary Search Trees are the most efficient in all functions of Access, Search, Deletion, and Insertion.

The Project

The project showing the use of a Binary Search Tree is a C++ application that takes a csv file and transfers it into a Binary Search Tree that contains the different Courses available at a school. There are functions to search for, insert, and delete elements along with the ability to print out the courses available. This shows the Binary Search Tree in action.

As part of this informatic, there is an attached example project using the Binary Search Tree along with examples of a Hash Table, Linked List, and Vector Sorting.